NoSQL Database Assignment

1. Compare RDBMS and NoSQL Databases

RDBMS stores data in tables with rows and columns. It usually follows a fixed schema, while NoSQL databases can use flexible schemas. RDBMS commonly scales vertically by increasing server power; NoSQL often scales horizontally by adding servers. RDBMS uses relational tables, whereas NoSQL supports key-value, document, column-family and graph models. RDBMS generally provides strong consistency, while some NoSQL systems allow eventual consistency. NoSQL can offer better performance for huge, distributed data. Examples: MySQL for banking and MongoDB for flexible product catalogues.

2. NoSQL Databases and Their Use Cases

MongoDB is a document database that stores data in flexible JSON-like documents. It is useful for content management, product catalogues and web applications. Apache Cassandra is a distributed column-family database designed for high availability and handling large amounts of data. It is used in messaging and IoT systems. Apache HBase is a column-family database built on Hadoop. It supports very large tables and is useful for log and sensor data. Neo4j is a graph database that stores nodes and relationships. It is used in social networks and recommendation systems.

3. RDBMS Approach and Its Challenges

The RDBMS approach stores information in related tables. Each table has rows and columns, and relationships are created using primary and foreign keys. SQL is used to insert, update and retrieve data. RDBMS provides a fixed schema, strong consistency and reliable transactions. However, traditional systems may face difficulty when data becomes extremely large or arrives very quickly. Scaling across many servers can be complex. Fixed schemas make frequent changes difficult, and joining huge tables can reduce performance. Distributed applications may also face network delays and availability challenges.

4. NoSQL Approach

NoSQL means “Not Only SQL”. It is an approach to storing and managing data without depending only on traditional relational tables. NoSQL databases support flexible data structures such as documents, key-value pairs, columns and graphs. They can scale horizontally by adding more servers, which helps manage large amounts of data. Many NoSQL systems distribute data across servers and use replication for availability. Their flexible schemas make changes easier, while suitable data models can improve speed. Examples include MongoDB, Cassandra, HBase and Neo4j.

5. NoSQL Data Models

A key-value model stores each value with a unique key, like a dictionary; Redis is an example and is useful for caching. A document model stores information in documents, often in JSON format; MongoDB can store a customer's name and address together. A column-family model groups related columns and supports large distributed data; Cassandra is an example. A graph model stores entities as nodes and connections as relationships; Neo4j can represent people and their friendships. Each model suits a different type of application and query.

6. Aggregate-Oriented Databases

Aggregate-oriented databases store related data together as one unit called an aggregate. An aggregate may contain a main record and its related information. For example, an online order can be stored with customer details, items and delivery information in one document. The database can read or update this unit together. This design can reduce the need for complex joins and improve performance in distributed systems. Document, key-value and column-family databases often use aggregate-oriented ideas. Developers must decide which information belongs together based on application needs.

7. MongoDB Document Model vs Cassandra Column-Family Model

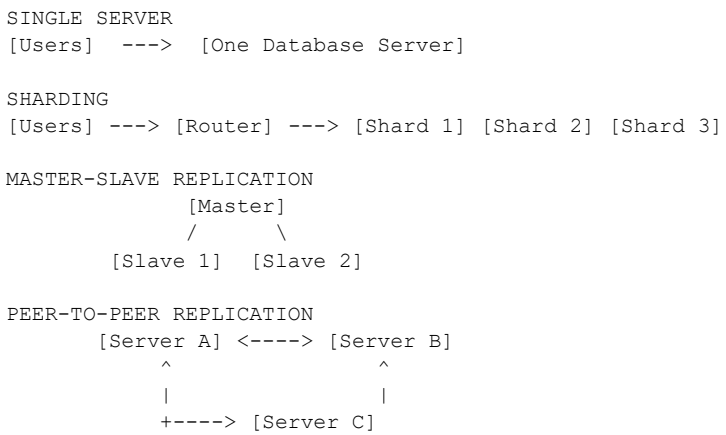
MongoDB stores data as documents, usually in BSON format, with fields and nested objects. Its structure is flexible, so different documents may contain different fields. It is suitable for applications such as an online shopping catalogue, where products have different attributes. Cassandra stores data in tables with rows and flexible columns grouped into column families. It is designed for high availability and large-scale write operations. It is suitable for applications such as recording time-stamped sensor readings or user activity across many servers.

8. Replication and Sharding

Replication means keeping copies of the same data on multiple servers. If one server fails, another copy may still be available. Sharding means splitting a large dataset into smaller parts and storing each part on a different server. This spreads storage and workload across machines. Replication mainly improves availability and fault tolerance, while sharding mainly improves capacity and scalability. Distributed NoSQL databases often use both techniques. Together, they help systems manage large amounts of data, serve many users and continue working when a server becomes unavailable.

9. Distribution Models

A single-server model stores and processes all data on one machine. In sharding, data is divided into parts and each server stores a different part. In master-slave replication, the master accepts updates and copies them to slave servers, which commonly handle read requests. In peer-to-peer replication, multiple servers can accept updates and share changes with one another. Single-server design is simple but has limited scale. Sharding increases capacity, while replication improves availability. Peer-to-peer systems can avoid one central write server but need conflict handling.



10. Master-Slave Replication

Master-slave replication uses one main server called the master and one or more slave servers. The master usually accepts write operations and sends data changes to the slaves. Slaves maintain copies of the master's data and may handle read requests. This can reduce the master's reading workload and provide backup copies. However, the master can become a bottleneck or a single point of failure. Replication delay may also cause slaves to show older data. It is useful for read-heavy websites, reporting systems and applications needing read copies.

11. Peer-to-Peer Replication

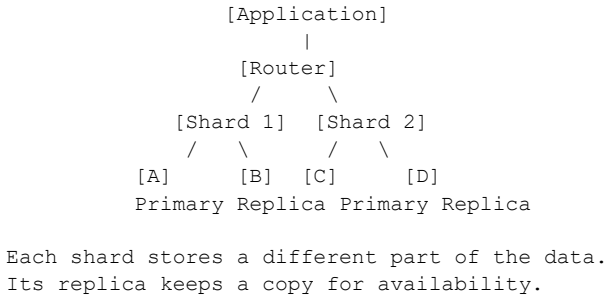
Peer-to-peer replication is a model in which multiple database servers have copies of data and can accept read and write operations. The servers share updates with one another so that their data becomes consistent. Unlike master-slave replication, there is no single master responsible for all writes. This can improve availability and distribute the workload. However, two servers may update the same data at the same time, creating conflicts. The system needs rules to resolve these conflicts. It is useful for distributed applications operating across multiple locations.

12. Sharding

Sharding is a technique that divides a large database into smaller parts called shards. Each shard is stored on a different server. Data may be divided using a key, such as customer ID or region. When a request arrives, the system uses the key to find the correct shard. Sharding increases storage capacity and spreads read and write work across servers. It can improve performance for large applications. Challenges include choosing a good shard key, balancing data, handling queries across shards and moving data when servers are added.

13. Combining Sharding and Replication

A distributed NoSQL database can use sharding and replication together. First, the database divides the complete dataset into shards. Each shard is stored on a group of servers, and replicas keep copies of that shard. For example, Shard 1 may be copied to Servers A and B, while Shard 2 is copied to Servers C and D. Requests are routed to the correct shard. Replicas provide backup and can serve reads. This design increases storage and processing capacity while improving availability if a server fails.



Each shard stores a different part of the data.
Its replica keeps a copy for availability.

14. MapReduce

MapReduce is a programming model used to process very large datasets across multiple computers. It has two main phases. In the Map phase, input records are read and converted into key-value pairs. In the Reduce phase, values with the same key are combined to produce a result. For example, to count student records by course, Map emits each course with the value 1. Reduce adds the values for each course. The final output shows the number of students in every course. This method supports parallel processing.

15. Role of MapReduce in Database Processing

MapReduce helps process large amounts of data by dividing a task into smaller jobs that can run on many computers. The Map phase reads records and creates intermediate key-value pairs. The Reduce phase groups and combines those pairs to produce final results. It is useful for counting, sorting, filtering and summarising data. For example, an online store can analyse millions of sales records to find total sales for each product. MapReduce supports parallel processing and can continue despite some machine failures, making it useful for big-data analysis.